ТЕХНОЛОГИИ ХРАНЕНИЯ БОЛЬШИХ ДАННЫХ

Хранение неструктурированных данных

Internet of Things



Термин «Интернет вещей» (IoT) впервые использовал в 1999 году британский новатор в области технологий по имени Кевин Эштон для описания системы, в которой предметы физического мира могут подключаться к Интернету с помощью датчиков.

Эштон создал этот термин для того, чтобы проиллюстрировать потенциальные возможности подключения меток радиочастотной идентификации (RFID), используемых в корпоративных цепочках поставок, для подсчета и отслеживания товаров без необходимости вмешательства со стороны человека.

Сегодня термин «Интернет вещей» широко используется для описания сценариев, в которых подключение к Интернету и вычислительные функции распространяются на ряд объектов, устройств, датчиков и других предметов повседневной жизни.

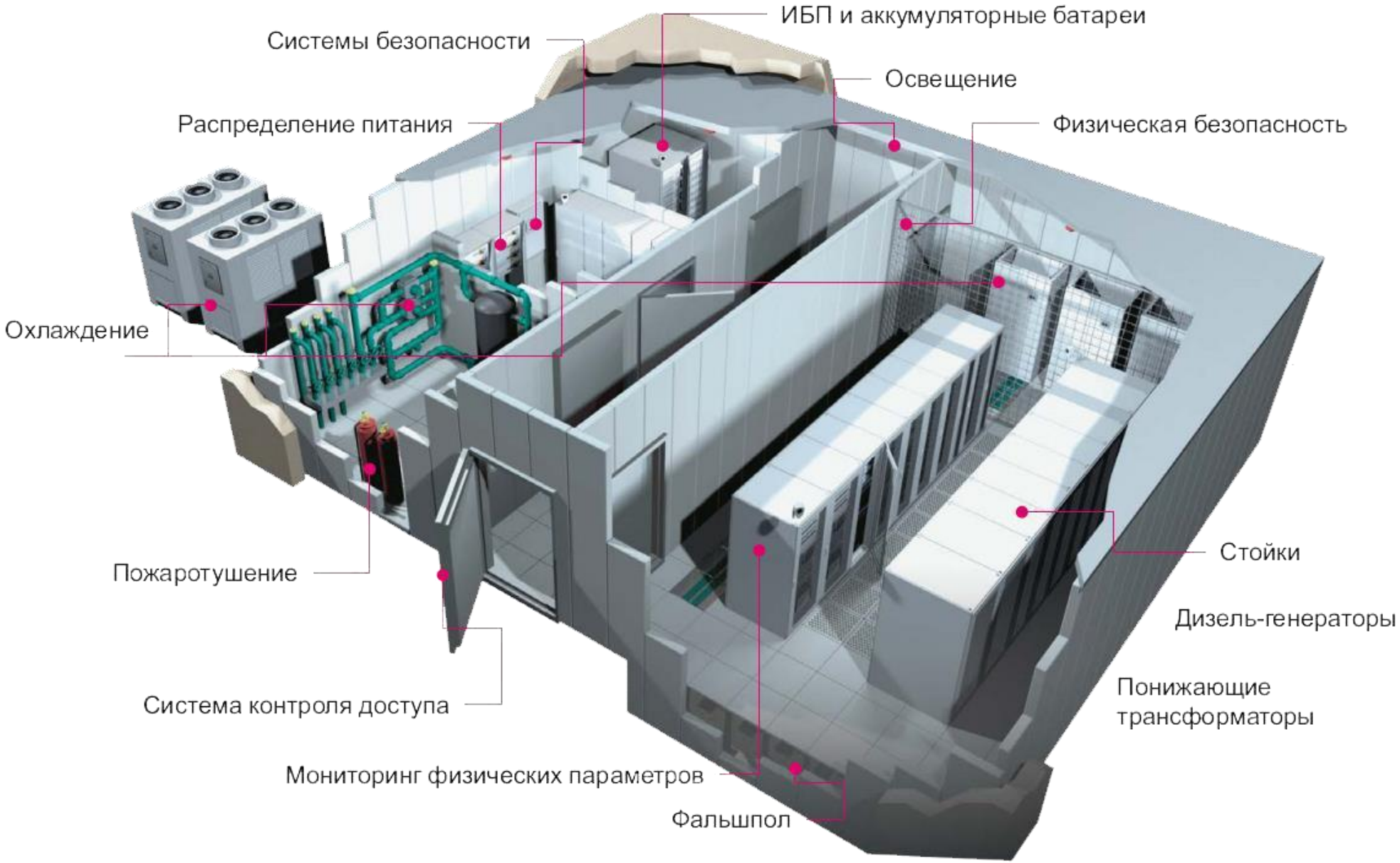
«Условия» для использования IoT

УСЛОВИЯ	ПРИМЕРЫ
ЧЕЛОВЕК Устройства, закрепленные на человеческом теле или внутри него	Устройства для мониторинга и поддержания здоровья, а также для обеспечения хорошего самочувствия людей; управление ходом заболевания, повышение уровня физической подготовки или производительности
ЖИЛЬЕ Здания, в которых живут люди	Домашние контроллеры и системы безопасности
ТОЧКИ РОЗНИЧНЫХ ПРОДАЖ Места, где потребители делают покупки	Магазины, банки, рестораны, арены — любые места, где люди принимают решения о покупках и делают их; кассы самообслуживания, специальные предложения, оптимизация товарных запасов
ОФИСЫ Места занятости работников умственного труда	Управление энергопотреблением и безопасностью в офисных зданиях; повышение производительности, в том числе, для мобильных сотрудников
ПРОИЗВОДСТВЕННЫЕ ПРЕДПРИЯТИЯ Стандартизованная производственная среда	Места с повторяющейся последовательностью рабочих операций, включая больницы и фермы; производственная эффективность, оптимизация использования оборудования и инвентаря

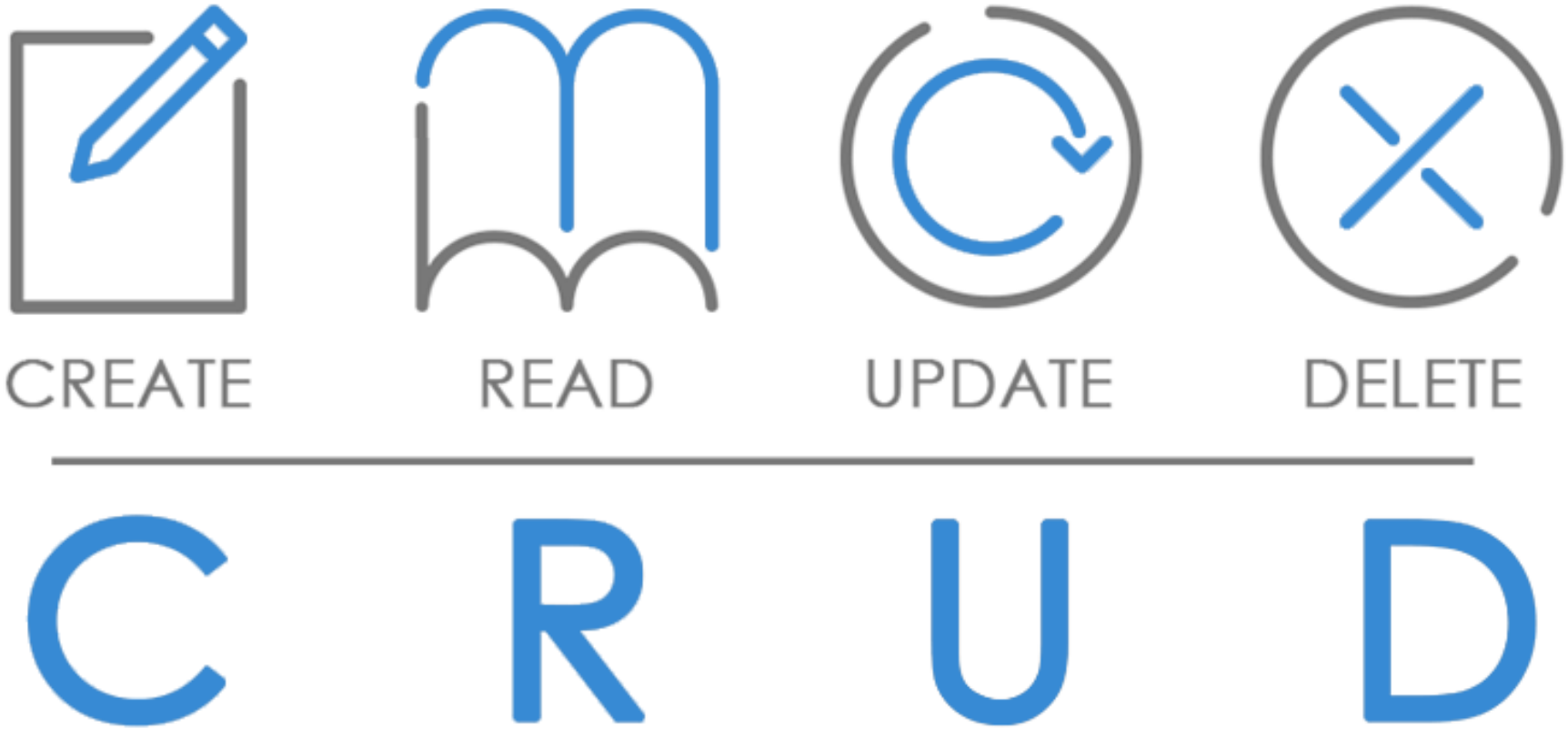
«Условия» для использования IoT

УСЛОВИЯ	ПРИМЕРЫ
РАБОЧИЕ ОБЪЕКТЫ Специализированная производственная среда	Горная промышленность, нефтегазовая промышленность, строительство; производственная эффективность, профилактическое обслуживание, здоровье и безопасность
ТРАНСПОРТНЫЕ СРЕДСТВА Системы внутри движущихся транспортных средств	Транспортные средства, включая автомобили, грузовики, суда, самолеты и поезда; обслуживание по техническому состоянию, конструкция на основе условий использования, предпродажный анализ
ГОРОДА Городская среда	Общественные места и инфраструктура в городских условиях; адаптивное регулирование движения транспорта, интеллектуальные счетчики, мониторинг окружающей среды, управление ресурсами
ОТКРЫТЫЕ ПРОСТРАНСТВА За пределами городской среды (и других условий)	В число открытых пространств входят железнодорожные пути, автономные транспортные средства (за пределами города) и аэронавигация; составление маршрута в реальном времени, услуги подключаемой навигации, отслеживание грузов

ЦОД – TIER - RAID



CRUD



ACID

В мире реляционных баз данных общеизвестны ACID-транзакции, некоторое время являвшиеся эталоном. Гарантии ACID обеспечивают безопасную среду для обработки данных:

- ***атомарность*** **(Atomic) - все операции в транзакции либо успешны, либо для всех них выполняется откат;**
- ***согласованность*** **(Consistent) - по окончании транзакции база данных является структурно согласованной;**
- ***изолированность*** **(Isolated) - транзакции не мешают друг другу. Спорные ситуации разрешаются базой данных так, что транзакции выполняются последовательно;**
- ***долговечность*** **(Durable) - результаты применения транзакции не должны теряться, даже при сбоях.**

Эти свойства означают, что сразу по завершении транзакции ее данные согласуются (так называемая *согласованность записи*) и записываются на диск (или на диски, или в разные области памяти). Это отличная абстракция для разработчиков приложений, но требует сложных блокировок, которые могут вызвать логическую недоступность, и, как правило, считается излишне тяжеловесным шаблоном в подавляющем большинстве прикладных областей.

NoSQL = Not Only SQL

Термин «NoSQL» впервые был использован в 1998 году для описания реляционной базы данных, не использовавшей SQL

Основные особенности:

- **Отказ от реляционной модели и языка SQL**
- **Использование распределенной архитектуры**
- **Отсутствие полноценной поддержки ACID-транзакций**

Преимущества:

- **Большая производительность**
- **Хорошая масштабируемость при возрастающих нагрузках и огромных объемах данных**

BASE

Для многих прикладных областей ACID-транзакции являются излишне пессимистичными. В мире NoSQL ACID-транзакции вышли из моды, поскольку хранилища смягчили требования к немедленной согласованности, актуальности данных и их точности, чтобы получить другие преимущества, такие как масштабируемость и гибкость. Вместо ACID возник другой популярный подход BASE, описывающий принципы более оптимистичной стратегии хранения:

- ***обычно доступно*** **(Basic Availability) - хранилище доступно большую часть времени;**
- ***гибкое состояние*** **(Soft-state) - хранилища не обязаны соблюдать очередность записей, и разные реплики не должны постоянно согласовываться;**
- ***отложенная согласованность*** **(Eventual Consistency) - хранилища достигают согласованности с некоторой задержкой по времени (например, позже, во время чтения).**

Принципы BASE значительно слабее гарантий ACID, и между ними нет прямого соответствия. Значения в BASE-хранилище доступны (потому что это является основой масштабирования), но это не предлагает гарантированной согласованности реплик при записи. BASE-хранилища обеспечивают менее строгий контроль: данные будут согласованы позднее, вероятнее всего, во время чтения, или всегда будут согласованы, но только для определенных фиксаций, обработанных последними.

Отсутствие жесткой структуры объектов БД

Schema-less данные – в NoSQL БД структура данных заранее не регламентируется. Вместо строго структурированных таблиц имеем, например, набор разнородных документов.



Relational data model



Document data model

Relational

Person:

Pers_ID	Surname	First_Name	City
0	Miller	Paul	London
1	Ortega	Alvaro	Valencia
2	Huber	Urs	Zurich
3	Blanc	Gaston	Paris
4	Bertolini	Fabrizio	Rom

no relation

Car:

Car_ID	Model	Year	Value	Pers_ID
101	Bentley	1973	100000	0
102	Rolls Royce	1965	330000	0
103	Peugeot	1993	500	3
104	Ferrari	2005	150000	4
105	Renault	1998	2000	3
106	Renault	2001	7000	3
107	Smart	1999	2000	2

MongoDB Document

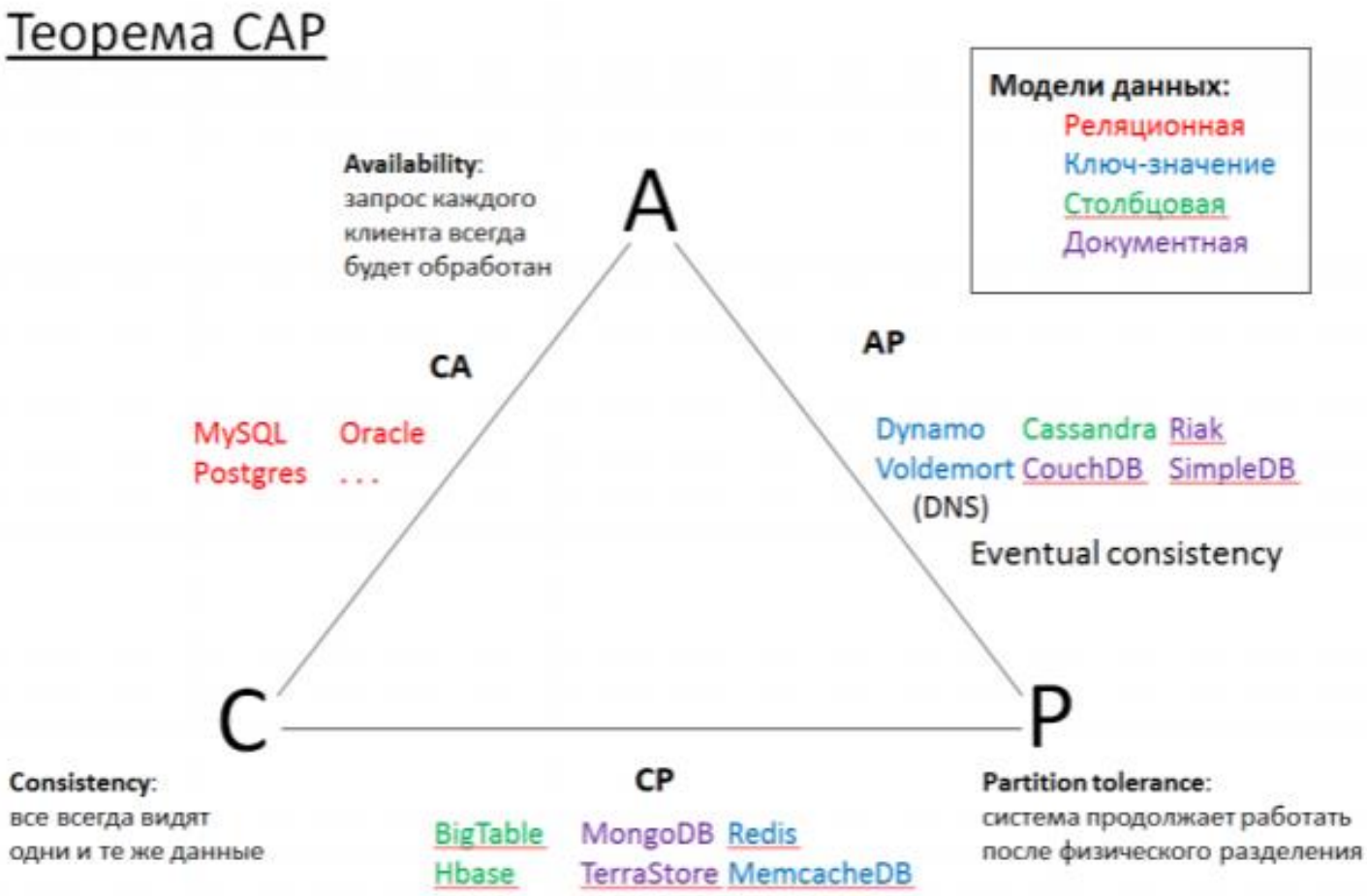
```
{
  first_name: 'Paul',
  surname: 'Miller'
  city: 'London',
  location: [45.123,47.232],
  cars: [
   { model: 'Bentley',
     year: 1973,
     value: 100000, ... },
   { model: 'Rolls Royce',
     year: 1965,
     value: 330000, ... }
  ]
}
```

Плюсы: структура каждого документа может быть уникальной (он сам себя описывает).
Минусы: поддержка структуры данных переносится с сервера в логику приложений.

Теорема CAP

Причины появления – распределенность систем:

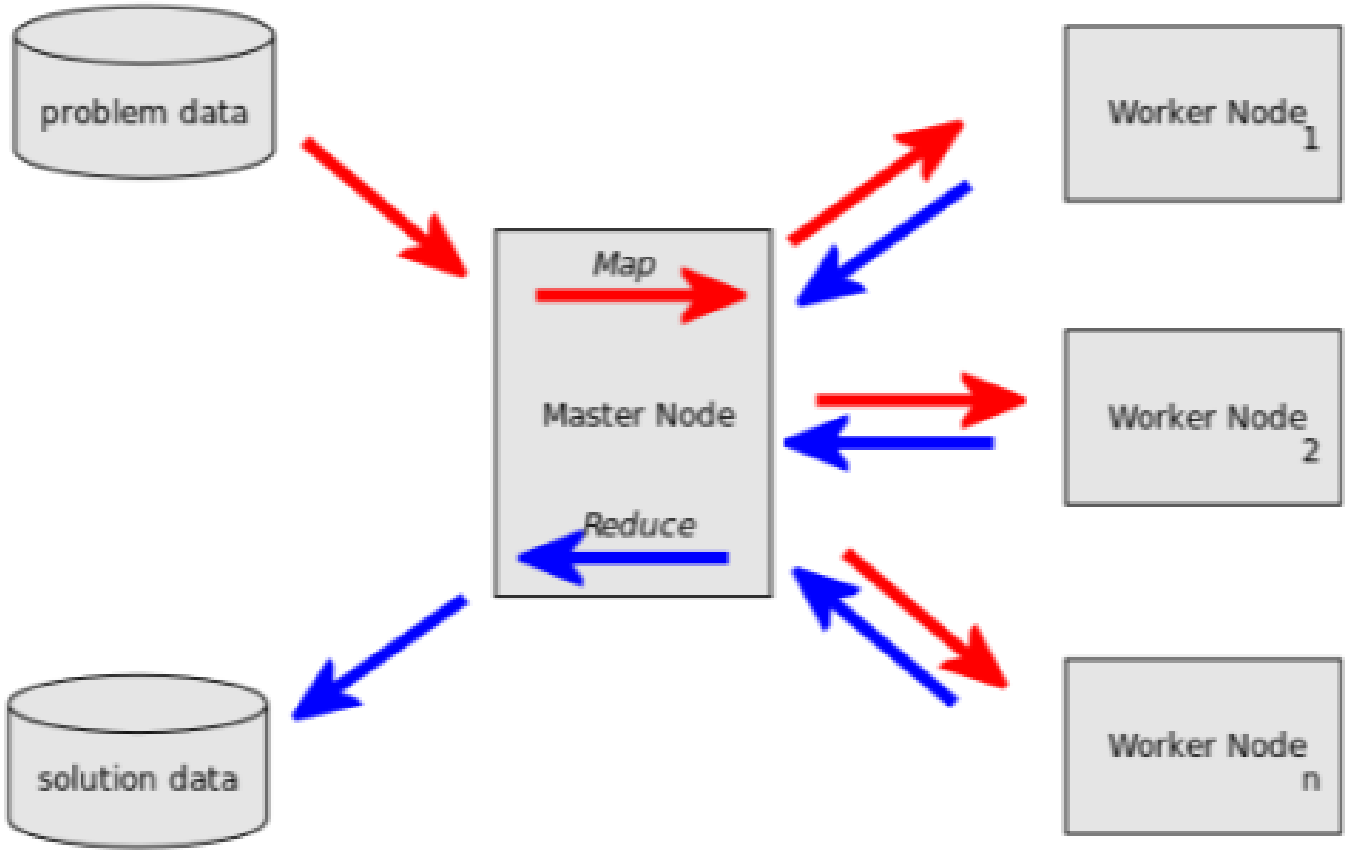
- **Потребность в горизонтальной масштабируемости БД (путем добавления новых узлов).**
- **Сложность эффективной реализации транзакций в распределенной среде.**



MapReduce

Механизм параллельной обработки больших данных на кластерах:

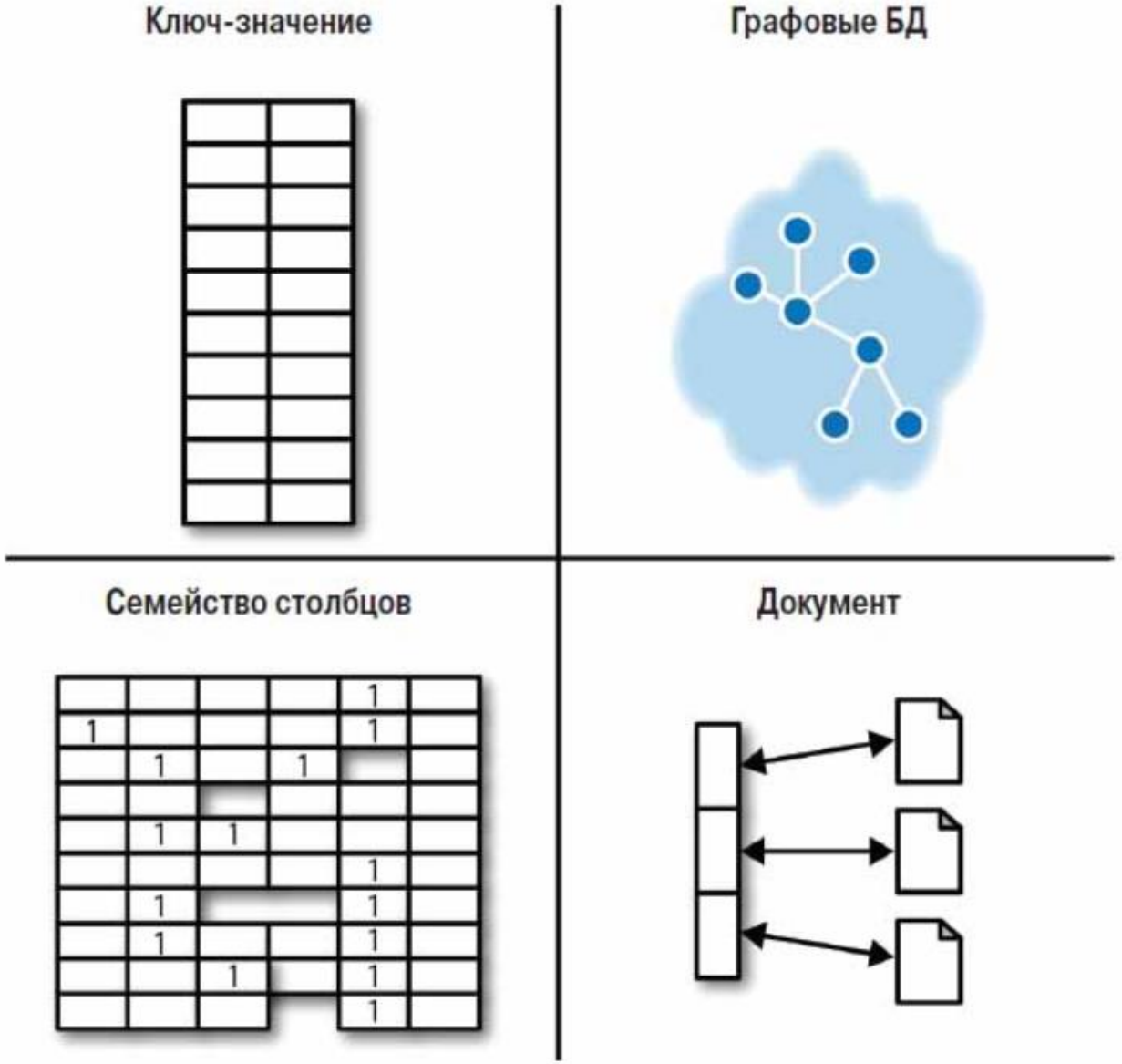
- **Map: Главный узел (master node) делит задачу на части, распределяя их по остальным машинам (worker nodes).**
- **Reduce: Master node получает предварительные результаты, и формирует из них конечный результат.**



Реляционные СУБД vs. NoSQL

	SQL	NoSQL
Хранилище данных	Таблицы (строки и столбцы). Строка - информация об одной сущности, а столбцы - отдельные значения данных.	Различными модели хранения данных (документ, граф, ключ-значение, столбчатые данные).
Схемы и гибкость	Каждая запись соответствует определенной схеме. Изменение схемы требует изменение всей базы данных.	Схема данных является динамической и может меняться в любой момент времени. Запись может содержать не все столбцы.
Масштабируемость	В основном по вертикали (на одном сервере). Масштабирование на нескольких серверах возможно, но сложно.	По горизонтали (на нескольких серверах). Технологии NoSQL могут автоматически распределять данные по серверам.
Соответствие ACID	Да.	Не полностью (ради производительности и масштабируемости).

Модели данных NoSQL



Модели данных NoSQL: Ключ - значение

Системы «ключ-значение»

- **CRUD-операции (Create-Read-Update-Delete)**
- **Хорошая горизонтальная масштабируемость**
- **Простота и производительность**
- **Плохо подходят для операций, отличных от CRUD**



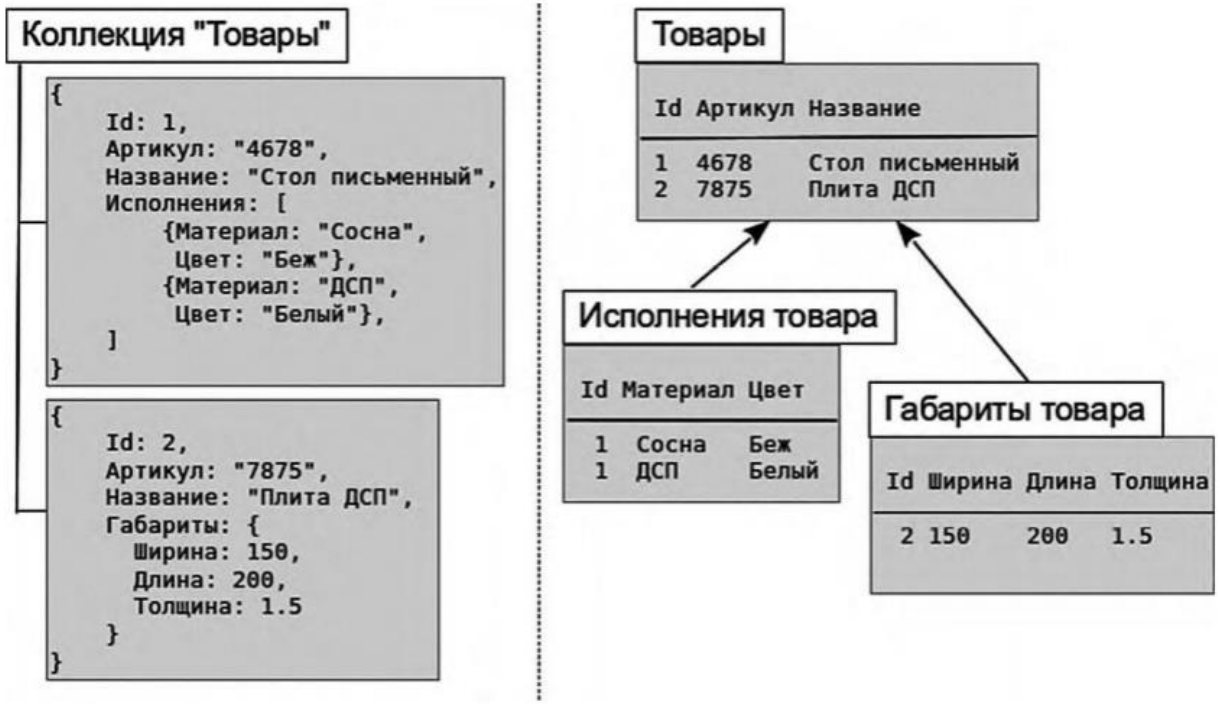




Модели данных NoSQL: Документ-ориентированная

Документ-ориентированные системы

- **Единица хранения – документ, т.е. объект, обладающий произвольным набором атрибутов (полей), который может быть представлен, например, в JSON**
- **Поддерживают поиск по полям документов, индексы, часто допускаются вложенные документы и массивы.**







Модели данных NoSQL: Big Table

Системы типа Google BigTable

- **Данные хранятся в виде строк. Для доступа к данным три ключа: ключ строки (по нему отсортированы строки в базе), ключ столбца, временная метка.**
- **Связанная информация хранится в одном месте.**







Строковое и колоночное хранение данных

Row-oriented: rows stored sequentially in a file

Key	Fname	Lname	State	Zip	Phone	Age	Sales
1	Bugs	Bunny	NY	11217	(123) 938-3235	34	100
2	Yosemite	Sam	CA	95389	(234) 375-6572	52	500
3	Daffy	Duck	NY	10013	(345) 227-1810	35	200
4	Elmer	Fudd	CA	04578	(456) 882-7323	43	10
5	Witch	Hazel	CA	01970	(567) 744-0991	57	250

Column-oriented: each column is stored in a separate file
Each column for a given row is at the same offset.

Key
1
2
3
4
5

Fname
Bugs
Yosemite
Daffy
Elmer
Witch

Lname
Bunny
Sam
Duck
Fudd
Hazel

State
NY
CA
NY
CA
CA

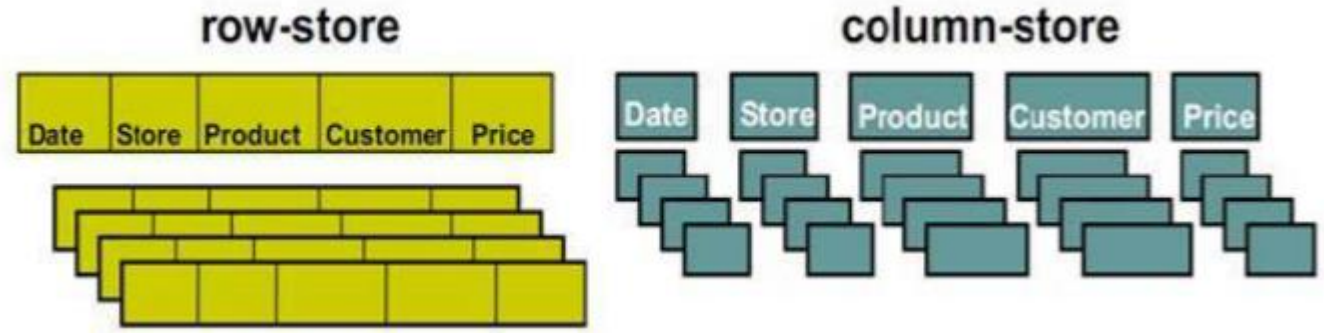
Zip
11217
95389
10013
04578
01970

Phone
(123) 938-3235
(234) 375-6572
(345) 227-1810
(456) 882-7323
(567) 744-0991

Age
34
52
35
43
57

Sales
100
500
200
10
250

Колоночное хранение данных



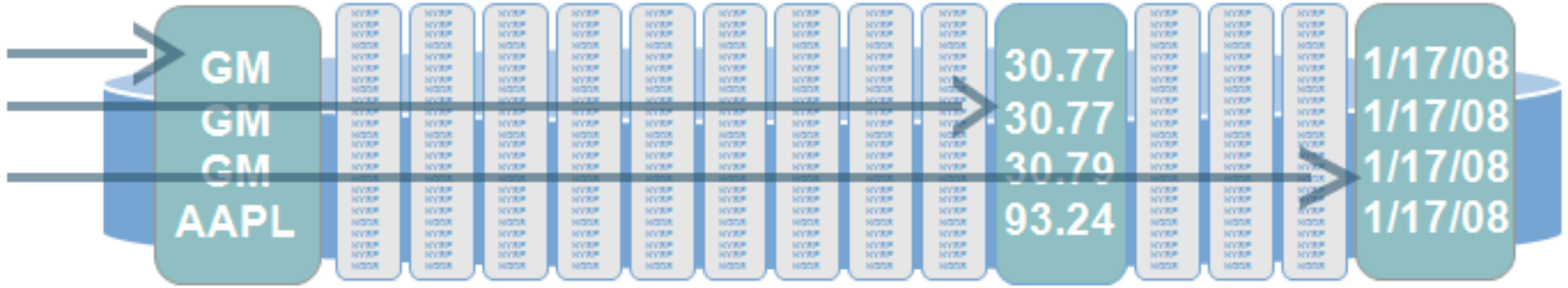
Под построчным хранением данных обычно понимается физическое хранение всей строки таблицы в виде одной записи, в которой поля идут последовательно одно за другим, а за последним полем записи в общем случае идет первое следующей записи. Приблизительно так:
[A1, B1, C1], [A2, B2, C2], [A3, B3, C3]…
где A, B и C — это поля (столбцы), а 1,2 и 3 — номер записи (строки).

Колоночное хранение - с точки зрения SQL-клиента данные представлены как обычно в виде таблиц, но физически эти таблицы являются совокупностью колонок, каждая из которых, по сути, представляет собой таблицу из одного поля. При этом физически на диске значения одного поля хранятся последовательно друг за другом — приблизительно так:
[A1, A2, A3], [B1, B2, B3], [C1, C2, C3] и т.д.

Колоночное хранение данных

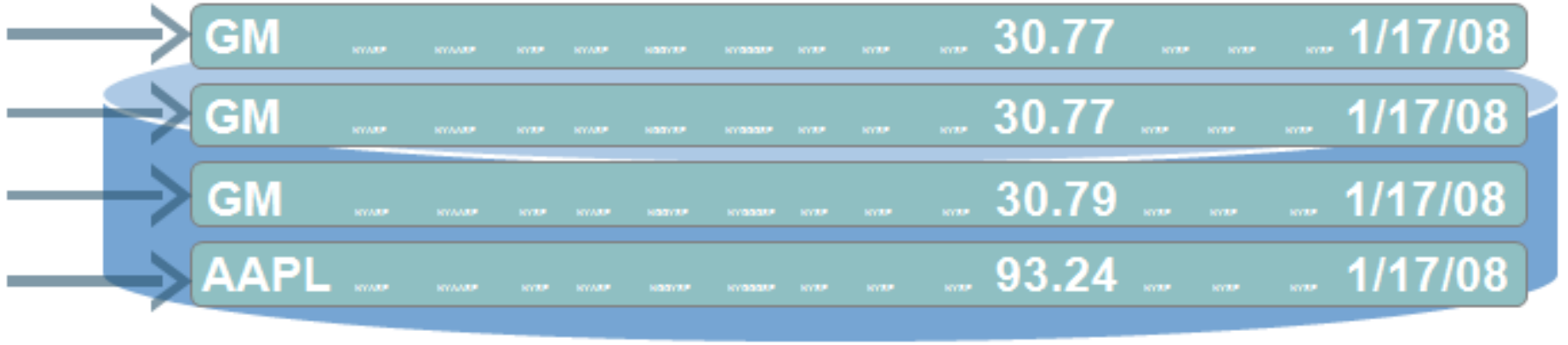
Хранение по столбцам

Чтение 3х столбцов

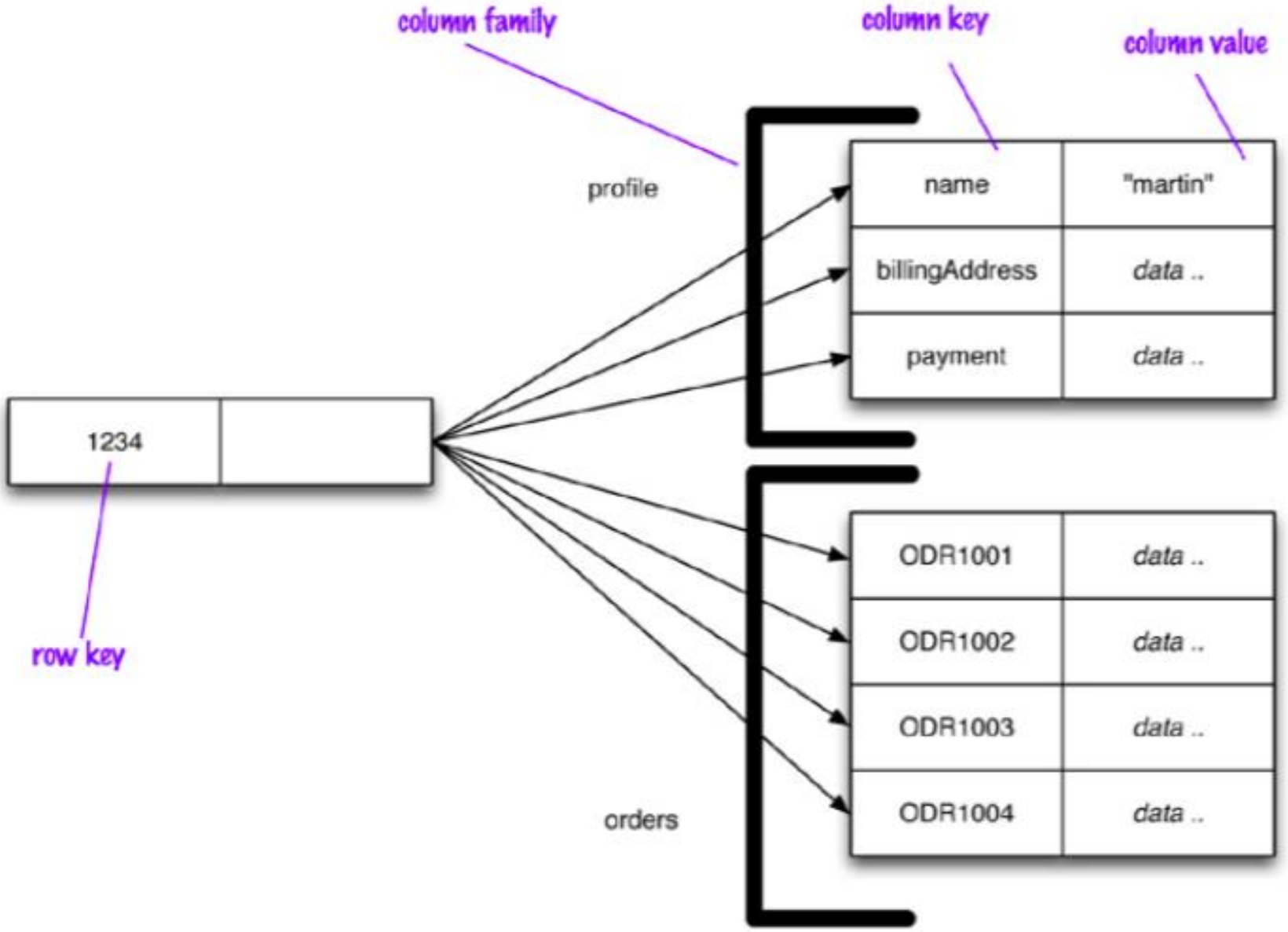


Хранение по строкам

Чтение всех столбцов



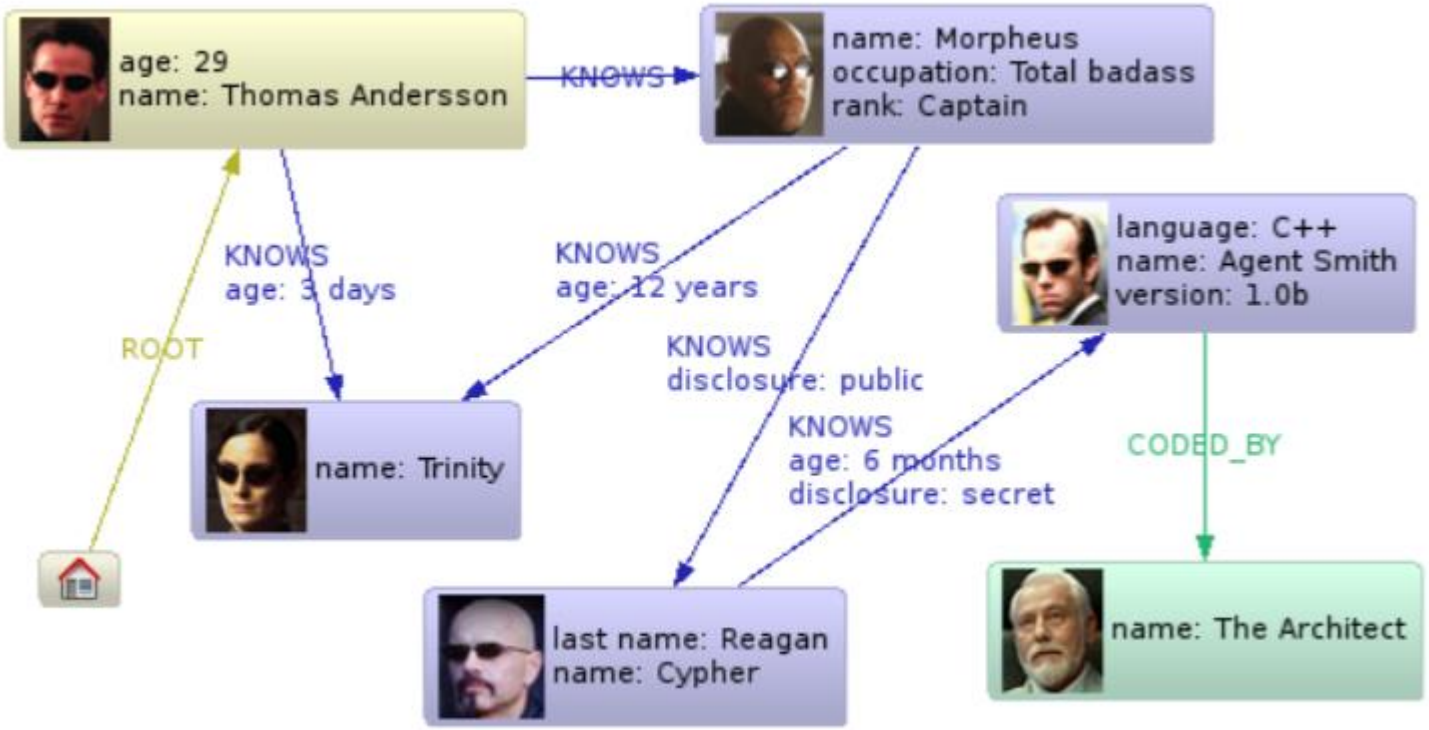
Column-family data model



Модели данных NoSQL: Графовая

Графовые системы:

- **Состоит из узлов и связей между ними.**
- **Как с узлами, так и со связями можно ассоциировать свойства (пары ключ-значение), в которых хранятся данные.**
- **Быстрый просмотр узлов и связей для поиска нужных данных.**



NoSQL СУБД

Document store
- Lotus Notes
- CouchDB
- MongoDB
- Apache Jackrabbit
- Colayer
- XML databases
  - o MarkLogic Server
  - o eXist

Graph
- Neo4j
- AllegroGraph

Tabular
- BigTable
- Mnesia
- Hbase
- Hypertable

Key/value store on disk
- Tuple space
- Memcachedb
- Redis
- SimpleDB
- flare
- Tokyo Cabinet
- BigTable

Key/value cache in RAM
- memcached
- Velocity
- Redis

Eventually-consistent key-value store
- Dynamo
- Cassandra
- Project Voldemort

Ordered key-value store
- NMDB
- Luxio
- Memcachedb
- Berkeley DB

Object database
- Db4o
- InterSystems Caché
- Objectivity/DB
- ZODB